17 · Responsible ML in Practice

> **Goal:** turn fairness, explainability, governance and uncertainty from slideware into measurable checks that run in the pipeline and survive retraining.

This chapter is orientation for engineers, not legal advice. Regulatory and fairness-law details change and differ by jurisdiction: verify current text with counsel and primary sources. Legacy governance mechanics live in Module 04 section 5 (legacy); risk tiers are in 4.15 and audit evidence in 10.16.

Table of Contents

- 17.1 Fairness Metrics, Trade-offs and Measuring Without Protected Attributes
- 17.2 Bias Mitigation: Pre-, In- and Post-Processing
- 17.3 Explainability in Production: SHAP, LIME, Reason Codes and Stability
- 17.4 Governance in Practice: Model Cards, Datasheets and Impact Assessments
- 17.5 Calibration, Uncertainty and Abstention

17.1 Fairness Metrics, Trade-offs and Measuring Without Protected Attributes

Conceptual Deep Dive

Fairness metrics compare model behaviour across groups. **Demographic parity** compares positive-prediction rates (loan approval rate by group). **Equal opportunity** compares true positive rates among the truly qualified; **equalized odds** also requires equal false positive rates. **Predictive parity** compares precision (PPV), and **calibration within groups** requires that a score of 0.7 means 70% in every group. Each answers a different harm: in lending, equal opportunity protects creditworthy applicants from rejection; in fraud, FPR parity limits who gets wrongly blocked; in hiring, parity of selection rates is often what regulators look at first.

Impossibility results (Kleinberg et al. 2016; Chouldechova 2017) show that when base rates differ, calibration within groups and equal error rates cannot both hold unless prediction is perfect. So choosing a metric is a decision, not a computation: use a harm model (who is hurt by false positives versus false negatives), base-rate differences, legal constraints and stakeholder input, then document the choice and its rationale in the model card (17.4).

Without protected attributes, you cannot measure what you cannot see. Options: collect them voluntarily for audit only, in a separated store with restricted access and legal review (10.15); use surrogate or inferred attributes (proxies such as geocoded name methods) knowing they add error and their own privacy risk; or audit through secure enclaves or a third party. Proxy-based audits understate gaps when the proxy is noisy, so report them as estimates.

Statistical care: subgroup estimates need confidence intervals (bootstrap), small groups give wide intervals, intersectional slices (race by age) multiply groups and invite multiple-comparison false alarms, so state which slices were pre-specified. **Over time**, group base rates drift and decisions create feedback loops (denied applicants never generate repayment data), so re-measure continuously (8.18).

Edge cases: proxy features (ZIP code) reintroduce protected attributes; subgroup samples too small; a fairness metric improves while total harm increases (for example by degrading outcomes for everyone).

Cross-Connection Scenarios

- Mitigation methods that act on these gaps are in 17.2.
- Fairness gates in promotion tie to risk tiers in 4.15.
- Per-slice monitoring in production is in 8.18; sensitive-attribute handling is in 10.15.
- Decision-centric credit modelling and reject inference are in 18.5.

Production Anti-Patterns

- Reporting one fairness metric with no rationale: the choice is arbitrary and indefensible when challenged.
- Removing the protected column and calling it fair ("fairness through unawareness"): proxies carry the signal and you lose the ability to measure the gap.
- One-time audit before launch only: base rates and populations drift after launch.

Minimal Working Code

Group metrics with bootstrap intervals on synthetic data with different base rates.

```
import numpy as np

def make_data(n=6000, seed=0):
    rng = np.random.default_rng(seed)
    g = rng.integers(0, 2, n)
    y = (rng.random(n) < np.where(g == 0, 0.30, 0.50)).astype(int)
    score = 0.35 * y + rng.normal(0.4, 0.2, n)        # one score model, both groups
    return g, y, score

def rate(v):
    return v.mean() if len(v) else np.nan

def gaps(g, y, yhat):
    return {"dp": rate(yhat[g == 1]) - rate(yhat[g == 0]),
            "tpr": rate(yhat[(g == 1) & (y == 1)]) - rate(yhat[(g == 0) & (y == 1)]),
            "fpr": rate(yhat[(g == 1) & (y == 0)]) - rate(yhat[(g == 0) & (y == 0)])}

g, y, s = make_data()
yhat = (s > 0.6).astype(int)
point = gaps(g, y, yhat)
rng = np.random.default_rng(1)
boots = [gaps(g[i], y[i], yhat[i]) for i in (rng.integers(0, len(g), len(g)) for _ in range(300))]
for k, v in point.items():
    lo, hi = np.percentile([b[k] for b in boots], [2.5, 97.5])
    print(f"{k}: {v:+.3f}  95% CI [{lo:+.3f}, {hi:+.3f}]")
assert point["dp"] > 0      # higher base-rate group is selected more often
```

The calibration versus equal-error-rate tension: a score calibrated within each group gives unequal error rates at one threshold, and equalising TPR then breaks predictive parity.

```
import numpy as np

rng = np.random.default_rng(0)
def group(n, a, b):
    p = rng.beta(a, b, n)                # true risk; the score IS the probability, so calibrated
    return p, (rng.random(n) < p).astype(int)

pa, ya = group(20000, 2, 5)              # lower base rate
pb, yb = group(20000, 5, 5)              # higher base rate

def calib(p, y):
    bins = np.digitize(p, [0.25, 0.5, 0.75])
    return [(round(float(p[bins == i].mean()), 2), round(float(y[bins == i].mean()), 2)) for i in range(4
) if (bins == i).any()]
```

```
def rates(p, y, t):
    yhat = p >= t
    return dict(tpr=float(yhat[y == 1].mean()), fpr=float(yhat[y == 0].mean()), ppv=float(y[yhat].mean())
)

print("calibration A (score, observed):", calib(pa, ya))
print("calibration B (score, observed):", calib(pb, yb))
ra, rb = rates(pa, ya, 0.4), rates(pb, yb, 0.4)
print("same threshold  A:", {k: round(v, 2) for k, v in ra.items()}, "B:", {k: round(
v, 2) for k, v in rb.items()})
ts = np.linspace(0.2, 0.9, 71)            # equalise TPR by moving B's threshold
tb = ts[np.argmin([abs(rates(pb, yb, t)["tpr"] - ra["tpr"]) for t in ts])]
rb2 = rates(pb, yb, tb)
print(f"B at TPR-matched threshold {tb:.2f}:", {k: round(v, 2) for k, v in rb2.items()})
assert abs(rb2["tpr"] - ra["tpr"]) < 0.02 and abs(rb2["ppv"] - ra["
ppv"]) > 0.05   # equal TPR, unequal PPV
```

Interview Angle

Q: Your lending model has equal approval rates across groups but different false negative rates. What do you do? **A:** Equal approval rates satisfy demographic parity but qualified applicants in one group are rejected more often. Revisit the harm model: for lending, equal opportunity is usually the more defensible target. Measure TPR gaps with bootstrap intervals, check proxies and label bias, then consider mitigation (17.2) after legal review. Trade-off: moving toward equal opportunity changes approval rates and possibly calibration. Check: TPR gap CI containing zero on a holdout and per-group calibration preserved.

Q: How do you audit fairness without storing protected attributes? **A:** Use a separated, access-controlled audit dataset with voluntarily collected attributes, joined only inside a secure environment, or use inferred surrogates and report results as estimates with error bars. Never feed the attributes to the model or serving path. Trade-off: surrogates are noisy and carry their own privacy and accuracy risks. Check: validate the surrogate against a labelled sample and re-run the audit every release.

17.2 Bias Mitigation: Pre-, In- and Post-Processing

Conceptual Deep Dive

Mitigation acts at three points. **Pre-processing** changes the data: reweighing (Kamiran and Calders) gives each group-label cell weight P(g)P(y)/P(g,y) so group and label look independent; resampling, relabelling suspect labels and repairing representations are alternatives. It fits when the data is the root cause (under-represented groups, biased labels) and keeps the model unmodified. **In-processing** changes training: constrained optimisation, fairness regularisers or adversarial debiasing; libraries such as Fairlearn and AIF360 implement several of these (check current docs for APIs). It usually gives the best accuracy-fairness frontier but ties you to supported model classes. **Post-processing** adjusts outputs: group-specific thresholds or score adjustments to equalise a chosen error rate; it needs no retraining and works on black-box models, but using protected attributes at decision time may be unlawful in some contexts, so get legal sign-off before shipping it.

Every method moves along an **accuracy-fairness frontier**. Pick the operating point with stakeholders using holdout and slice metrics, not the training set; tuning a mitigation on the data used to evaluate it overstates the gain. Report intersectional slices too, because fixing group A versus B can shift harm to a subgroup such as young members of A. Threshold adjustment equalises error rates but breaks calibration of the decision threshold per group.

Data collection is also mitigation: targeted labelling, active sampling of under-represented slices and fixing coverage gaps often beat algorithmic fixes.

Operationalise: the mitigation must live in the training pipeline, not a notebook; add a fairness gate in CI and promotion (4.15), re-measure after every retrain (new data can reintroduce the gap), and monitor slices in production (8.18). Edge cases: mitigation shifts harm to another subgroup; thresholds adjusted without recalibration; evaluation leakage. As a rule of thumb, first confirm the gap is real with confidence intervals (17.1), then try the cheapest intervention that addresses the diagnosed cause, and record the chosen operating point and its accuracy cost in the model card (17.4) so later retrains can be compared against it.

Cross-Connection Scenarios

- Metric choice comes from 17.1; documentation of the chosen trade-off belongs in 17.4.
- Regression tests that pin fairness gaps are in 13.5.

- Retraining pipelines (chapter 5) must re-run the mitigation step.
- Threshold economics are in 17.5 and 18.2.

Production Anti-Patterns

- Mitigating in the notebook but not in the pipeline: the next retrain silently loses the fix.
- Applying group thresholds without legal sign-off: a technical fix that creates legal exposure.
- Ignoring intersectional groups: aggregate parity hides worse outcomes for sub-populations.

Minimal Working Code

Reweighing, then a weighted logistic regression compared with an unweighted one.

```
import numpy as np

def reweigh(g, y):
    w = np.ones(len(y))
    for gv in (0, 1):
        for yv in (0, 1):
            cell = (g == gv) & (y == yv)
            w[cell] = (g == gv).mean() * (y == yv).mean() / cell.mean()
    return w

def fit_logreg(X, y, w, iters=800, lr=0.5):
    X1 = np.c_[X, np.ones(len(X))]
    beta = np.zeros(X1.shape[1])
    for _ in range(iters):
        p = 1 / (1 + np.exp(-X1 @ beta))
        beta -= lr * X1.T @ (w * (p - y)) / w.sum()
    return beta

def make(n, seed):
    rng = np.random.default_rng(seed)
    g = rng.integers(0, 2, n)
    y = (rng.random(n) < np.where(g == 0, 0.25, 0.55)).astype(int)
    X = np.c_[y + rng.normal(0, 1.2, n), g + rng.normal(0, 0.5, n)]   # col 1 is a group proxy
    return X, y, g

def dp_gap(beta, X, g):
    s = np.c_[X, np.ones(len(X))] @ beta
    yhat = s > np.quantile(s, 0.65)              # select the top 35%
    return yhat[g == 1].mean() - yhat[g == 0].mean()

Xtr, ytr, gtr = make(8000, 0)
Xte, yte, gte = make(8000, 1)                    # holdout for evaluation
b_plain = fit_logreg(Xtr, ytr, np.ones(len(ytr)))
b_rw = fit_logreg(Xtr, ytr, reweigh(gtr, ytr))
print("DP gap plain:", round(dp_gap(b_plain, Xte, gte), 3), " reweighed:", round(dp_g
ap(b_rw, Xte, gte), 3))
assert abs(dp_gap(b_rw, Xte, gte)) < abs(dp_gap(b_plain, Xte, gte))
```

Post-processing: per-group thresholds to equalise TPR, tuned on one half and evaluated on the other, with the accuracy change reported (it can be a cost or, when scores are shifted per group as here, a small gain).

```
import numpy as np

rng = np.random.default_rng(0)
n = 20000
g = rng.integers(0, 2, n)
y = (rng.random(n) < np.where(g == 0, 0.3, 0.5)).astype(int)
s = 0.9 * y + rng.normal(0, 0.8, n) - 0.4 * g          # group 1 scores are shifted down
tune, test = np.arange(n) < n // 2, np.arange(n) >= n // 2

def acc(thr_by_group, mask):
```

```
    yhat = s >= np.where(g == 1, thr_by_group[1], thr_by_group[0])
    return (yhat == y)[mask].mean()

def tpr(thr, grp, mask):
    m = mask & (g == grp) & (y == 1)
    return (s[m] >= thr).mean()

grid = np.linspace(-1, 2, 301)
global_t = grid[np.argmax([acc((t, t), tune) for t in grid])]
target = tpr(global_t, 0, tune)                    # match group 0's TPR at the global threshold
t1 = grid[np.argmin([abs(tpr(t, 1, tune) - target) for t in grid])]
for name, thr in [("global", (global_t, global_t)), ("group-specific", (global_t, t1)
)]:
    print(f"{name:15s} test acc={acc(thr, test):.3f}  TPR g0={tpr(thr[0], 0, test):.3f}  TPR g1={tpr
(thr[1], 1, test):.3f}")
assert abs(tpr(t1, 1, test) - tpr(global_t, 0, test)) < abs(tpr(global_t, 1, test) - tpr(global_t, 0,
test))
```

Interview Angle

Q: Compare pre-, in- and post-processing mitigation. Which would you try first and why? **A:** I would diagnose first: if the gap stems from data coverage or labels, pre-processing (reweighing, better data) is cheapest and keeps one model for all. In-processing gives the best frontier but needs supported learners and tuning. Post-processing is fastest and model-agnostic but may require protected attributes at decision time, which needs legal review. Trade-off: accuracy cost versus operational and legal risk. Check: gap and accuracy on a holdout, plus intersectional slices.

Q: How do you keep a mitigation from silently regressing after retraining? **A:** Make it a pipeline step, not a notebook one-off, and add a fairness gate to promotion that compares gaps with the champion and fails on regression beyond a tolerance. Log per-group metrics with each run and monitor slices in production. Trade-off: gates can block releases on noisy small-group estimates, so gate on confidence intervals. Check: gap CI per release and alerts when production slice metrics diverge.

17.3 Explainability in Production: SHAP, LIME, Reason Codes and Stability

Conceptual Deep Dive

Local explanations describe one prediction; **global** ones describe model behaviour overall. **Model-agnostic** methods treat the model as a function: permutation importance (global), Shapley-value methods such as SHAP (local, additive attributions) and LIME (local linear surrogate fitted on perturbed samples). **Model-specific** ones use structure: tree paths, linear coefficients, TreeSHAP (exact and efficient for tree ensembles). Partial dependence and ICE curves show average and per-instance effect of one feature.

The pitfalls decide whether explanations are trustworthy. With **correlated features**, importance splits or hides across them, and perturbation creates unrealistic points the model never saw (extrapolation). Attributions depend on the **background data** (the reference distribution for "absent" features), so a changed background changes attributions. Near-identical inputs can receive different explanations (instability), sampling-based methods differ run to run, and attributions describe the model, **not causal effects** in the world: telling a user "raising your income by X changes the decision" requires a causal claim the explanation does not support.

Reason codes for adverse decisions map top contributing features to human-readable reasons ("high debt-to-income"). Requirements vary by jurisdiction (verify current rules). They must be consistent with the decision threshold, use the same model that made the decision, and be validated by reviewers.

Serving: exact Shapley values are exponential in features, so use TreeSHAP or sampling, precompute for batch decisions, cache for repeated inputs, compute asynchronously off the critical path, and log the explanation with the decision for audit (10.16). Exposing detailed attributions through a public API aids model extraction (10.15 for privacy, Module 10 section 1 (legacy)).

Testing: stability tests (perturb slightly, compare top-k), sanity checks such as the randomised-model test (explanations should change when weights are randomised), agreement with domain constraints (monotonic features). **Monitoring**: explanation drift (shift in mean attributions) is an early signal of changed behaviour (8.18).

Edge cases: explanations differ between runs because of sampling (fix seeds, increase samples); background drift; synchronous computation blowing the latency budget.

Cross-Connection Scenarios

- Reason codes support contestability in 10.16 and the governance artifacts in 17.4.
- Monotonic constraints and scorecards in credit are in 18.5.
- Latency budgets for explanation serving follow chapter 6.
- Attribution stability tests belong in 13.5.

Production Anti-Patterns

- Treating SHAP values as causal effects: users get wrong advice and regulators get wrong claims.
- Exposing detailed explanations to adversaries: attributions leak decision boundaries and ease extraction.
- Different explanation model than decision model: explanations describe a model that did not decide.

Minimal Working Code

Permutation importance with repeats and standard error, compared with linear coefficients when two features are noisy copies of one signal.

```
import numpy as np

rng = np.random.default_rng(0)
n = 2000
z = rng.normal(size=n)                          # latent factor seen through two noisy sensors
x1, x2 = z + 0.3 * rng.normal(size=n), z + 0.3 * rng.normal(size=n)
x3 = rng.normal(size=n)
X = np.c_[x1, x2, x3]
y = z + 0.5 * x3 + 0.3 * rng.normal(size=n)

coef = np.linalg.lstsq(np.c_[X, np.ones(n)], y, rcond=None)[0]
predict = lambda M: np.c_[M, np.ones(len(M))] @ coef
base_mse = np.mean((y - predict(X)) ** 2)

def perm_importance(cols, repeats=20):
    deltas = []
    for _ in range(repeats):
        Xp = X.copy()
        idx = rng.permutation(n)
        Xp[:, cols] = Xp[idx][:, cols]          # permute the columns jointly
        deltas.append(np.mean((y - predict(Xp)) ** 2) - base_mse)
    return np.mean(deltas), np.std(deltas, ddof=1) / np.sqrt(repeats)

for cols, name in [([0], "x1"), ([1], "x2"), ([0, 1], "x1+x2 together"), ([
2], "x3")]:
    m, se = perm_importance(cols)
    print(f"{name:15s} perm_importance={m:.3f} +/- {se:.3f}")
print("coefficients:", coef[:3].round(2))
assert abs(coef[0] + coef[1] - 1.0) < 0.3        # the effect is shared between the correlated pair
```

Exact Shapley values by enumeration versus a Monte Carlo permutation approximation.

```
import itertools, math
import numpy as np

f = lambda x: 3 * x[0] + 2 * x[1] * x[2] + x[3] ** 2
x = np.array([1.0, 2.0, 1.5, 2.0])
base = np.zeros(4)                              # background: one reference point
v = lambda S: f(np.where(np.isin(np.arange(4), list(S)), x, base))

def exact():
    phi = np.zeros(4)
    for i in range(4):
        others = [j for j in range(4) if j != i]
        for r in range(4):
            for S in itertools.combinations(others, r):
```

```
            w = math.factorial(r) * math.factorial(3 - r) / math.factorial(4)
            phi[i] += w * (v(S + (i,)) - v(S))
    return phi

def monte_carlo(n_perm, seed=0):
    rng = np.random.default_rng(seed)
    phi = np.zeros(4)
    for _ in range(n_perm):
        S = []
        for i in rng.permutation(4):
            phi[i] += v(S + [i]) - v(S)
            S.append(i)
    return phi / n_perm

ex = exact()
print("exact:", ex.round(3), " sum =", ex.sum().round(3), " f(x)-f(base) ="
, f(x) - f(base))
for n in (10, 100, 2000):
    print(n, "perms, max abs error:", np.abs(monte_carlo(n) - ex).max().round(3))
assert abs(ex.sum() - (f(x) - f(base))) < 1e-9
```

Interview Angle

Q: How would you produce reason codes for a declined applicant and how do you validate them? **A:** Compute attributions with the same model that made the decision (TreeSHAP for trees), take the top features pushing toward decline, map each to a vetted reason text and de-duplicate correlated ones. Validate with stability tests, a randomised-model sanity check, domain review of the mapping and a check that removing the top reason moves the score toward approval. Trade-off: fidelity versus readability. Check: top-3 reason stability under small input perturbation and reviewer agreement rate.

Q: Why can feature importance be misleading with correlated features? **A:** Credit is split arbitrarily between correlated features, and permuting one of them leaves the other carrying the signal, so each looks less important than the pair. Permutation also creates unrealistic rows. Linear coefficients become unstable. Mitigate with grouped importance, conditional methods, or dropping redundant features. Trade-off: groups lose granularity. Check: importance of the group versus individuals, and stability of rankings across bootstrap resamples.

17.4 Governance in Practice: Model Cards, Datasheets and Impact Assessments

Conceptual Deep Dive

Governance artifacts make decisions reviewable. A **model card** (Mitchell et al. 2019) documents intended use, out-of-scope use, training and evaluation data, metrics including per-slice results, limitations, ethical considerations and maintenance. **Datasheets for datasets** (Gebru et al. 2018) record why data was collected, composition, collection process, labelling, known gaps and permitted uses. An **impact assessment** analyses potential harms and mitigations before launch, and a **risk register** tracks open risks with owners. The model owner maintains the card, the data owner the datasheet, and risk or compliance reviews the assessment.

Automate documentation: populate cards from registry metadata, eval reports and slice tables; version the card with the model; and add a CI check that required fields and evaluation slices exist (4.15). Cards that are hand-written once go stale because metrics are computed on old data and the product's use drifts from the stated intent.

Review boards and RACI: define when review is required (by risk tier), what evidence is expected, who decides, who is consulted, how decisions are recorded and how teams appeal. A board that reviews everything becomes a bottleneck; tier the load and set a service level for decisions.

Risk classification: regulations such as the EU AI Act use risk tiers with escalating obligations (verify current text, scope and dates against primary sources). Map external categories onto internal tiers (4.15) so one workflow serves both.

Human oversight: define where a person reviews or can override, how escalation works and what is logged. Measure **override rates** by reviewer and segment: near-zero overrides suggest automation bias, very high ones suggest a weak model or unclear policy, and big differences between reviewers suggest inconsistent standards.

Incident handling for harms: intake channel, severity classification, authority to roll back or disable, user communication, and a learning loop into the card and risk register (11.17).

Edge cases: card describes the intended use but product deploys the model elsewhere; card metrics computed on stale data; board turns into a bottleneck.

Cross-Connection Scenarios

- Tiers and approvals from 4.15; evidence retention and audit packs from 10.16.
- Fairness results for the card come from 17.1 and 17.2; uncertainty and abstention policy from 17.5.
- Incident process links to 11.17.
- Registry metadata used for generation is in 4.16.

Production Anti-Patterns

- Model cards written once at launch: they describe a model that no longer exists.
- Governance disconnected from CI/CD: reviews happen in meetings while the pipeline ships anything.
- Documentation that lists no limitations: it reads as marketing and hides known risks.

Minimal Working Code

A model-card generator with a validator that fails on missing sections or slices.

```
REQUIRED = ["intended_use", "out_of_scope", "
training_data", "metrics", "slices", "limitations"]

def validate_card(meta: dict) -> list:
    errs = [f"missing section: {k}" for k in REQUIRED if not meta.get(k)]
    if meta.get("slices") is not None:
        for need in meta.get("required_slices", []):
            if need not in meta["slices"]:
                errs.append(f"missing evaluation slice: {need}")
    return errs

def render_card(meta: dict) -> str:
    errs = validate_card(meta)
    if errs:
        raise ValueError("; ".join(errs))
    lines = [f"# Model card: {meta['name']} v{meta['version']}",
             f"Intended use: {meta['intended_use']}", f"Out of scope: {meta['out_of_scope'
]}",
             f"Training data: {meta['training_data']}", "## Metrics"]
    lines += [f"- {k}: {v}" for k, v in meta["metrics"].items()]
    lines += ["## Slices"] + [f"- {k}: {v}" for k, v in meta["slices"].item
s()]
    lines += ["## Limitations"] + [f"- {x}" for x in meta["limitations"]]
    return "\n".join(lines)

meta = {"name": "credit", "version": 7, "intended_use": "pre
-screen applications for review",
        "out_of_scope": "final decisions without human review", "training_data&q
uot;: "synthetic 2025 snapshot",
        "metrics": {"auc": 0.81}, "slices": {"age<30": {"
auc": 0.77}},
        "required_slices": ["age<30", "region=north"], "limitations
": ["thin-file applicants"]}
print(validate_card(meta))
meta["slices"]["region=north"] = {"auc": 0.8}
print(render_card(meta).splitlines()[0])
```

An override-rate monitor for human-in-the-loop decisions.

```
import math
from collections import defaultdict
```

```
# log schema: one dict per reviewed decision
# {"reviewer": str, "segment": str, "model_decision": str, "human_deci
sion": str}

def override_report(log: list, z_alert: float = 3.0, min_n: int = 30) -> list:
    flags_all = [r["model_decision"] != r["human_decision"] for r in log]
    alerts = []
    for key in ("reviewer", "segment"):
        groups = defaultdict(list)
        for r in log:
            groups[r[key]].append(r["model_decision"] != r["human_decision"])
        for name, flags in groups.items():
            n, rate = len(flags), sum(flags) / len(flags)
            rest = (sum(flags_all) - sum(flags)) / max(len(log) - n, 1)   # everyone else
            se = math.sqrt(rest * (1 - rest) / n)
            if n >= min_n and se > 0 and abs(rate - rest) / se > z_alert:
                alerts.append((key, name, round(rate, 2), round(rest, 2)))
    return alerts

log = []
for i in range(300):
    reviewer = ["ana", "raj", "li"][i % 3]
    overridden = (i % 10 == 0) or (reviewer == "li" and i % 2 == 0)      # li overrides far mo
re
    log.append({"reviewer": reviewer, "segment": "retail", "model_deci
sion": "decline",
                "human_decision": "approve" if overridden else "decline"})
print(override_report(log))
assert any(a[1] == "li" for a in override_report(log))
```

Interview Angle

Q: What belongs in a model card and how do you stop it going stale? **A:** Intended and out-of-scope use, data summary, evaluation including per-slice metrics, limitations, ethical considerations, owner and version. Generate it from registry metadata and eval reports on every release and fail CI when required fields or slices are missing. Trade-off: generated text can be formulaic, so keep a human-written limitations section reviewed each release. Check: card version equals model version and evaluation date is within the agreed freshness window.

Q: Design the governance process for a new high-risk model with a two-week deadline. **A:** Classify the tier on day one, assign an independent validator, and run the evidence work in parallel: datasheet, impact assessment, fairness and slice report, explanation spec, rollback plan, human-oversight design. Use automated gates for completeness and a single board review with a pre-read. Trade-off: speed versus depth, so I would narrow scope (shadow or limited rollout) rather than skip evidence. Check: all gates green, board decision recorded, rollback tested.

17.5 Calibration, Uncertainty and Abstention

Conceptual Deep Dive

A model is **calibrated** if among predictions of 0.8, about 80% are positive. Check with reliability diagrams and summary scores: the Brier score and expected calibration error (ECE, a binned gap between confidence and accuracy; bin choice matters). Fix with **temperature scaling** (one parameter, preserves ranking), **Platt scaling** (logistic fit on scores), or **isotonic regression** (flexible, needs more data and can overfit). Calibrate on data separate from training, per segment when segments behave differently, and recalibrate after retraining or drift. Raw softmax or boosted-tree scores are rarely calibrated risks.

Uncertainty has two kinds: aleatoric (irreducible noise) and epistemic (lack of knowledge, reducible with data). Estimate with ensembles, MC-dropout, quantile regression, or **conformal prediction** (Vovk et al.; Angelopoulos and Bates tutorial 2021), which wraps any model and gives distribution-free coverage guarantees under exchangeability: calibrate a quantile of residuals or nonconformity scores on held-out data and widen predictions accordingly.

Abstention and deferral: a reject option routes low-confidence cases to a human or a fallback model. Plot **coverage versus risk**: as you answer fewer cases, error among answered cases should fall. Choose the threshold by expected cost (cost of an error versus cost of review), not by a default of 0.5.

Operational uses: cascades (cheap model first, expensive model on uncertain cases), active learning (label the uncertain), alerts on uncertainty drift, and canary guardrails (abort a rollout if confidence collapses).

Pitfalls: calibration degrades under distribution shift, conformal coverage assumes exchangeability and fails under shift, and out-of-distribution inputs often receive overconfident scores. Calibration and abstention also need ownership: someone must own the calibration set, the refresh cadence and the escalation capacity, otherwise the policy decays silently. Edge cases: the calibration set differs from production; abstention concentrates on a minority slice, shifting the burden to a group (measure abstention rate per group as in 17.1).

Cross-Connection Scenarios

- Abstention fairness is a 17.1 measurement; the human-review loop is a 17.4 design.
- Threshold economics for fraud and uplift are in 18.2 and 18.5.
- Drift monitors that trigger recalibration are in chapter 8.
- Cascades and fallback behaviour are in Module 11 section 7 (legacy).

Production Anti-Patterns

- Using raw probabilities as calibrated risks: decisions and expected-cost math are systematically off.
- Fixed 0.5 threshold regardless of costs: the optimum depends on the error cost ratio.
- Never measuring abstention fairness: the review burden or the missed service lands on one group.

Minimal Working Code

Platt-style calibration (fitted by Newton steps) with ECE before and after and a reliability table.

```
import numpy as np

rng = np.random.default_rng(0)
sig = lambda z: 1 / (1 + np.exp(-z))
logit = lambda p: np.log(p / (1 - p))

def data(n):
    z = rng.normal(0, 1.5, n)
    y = (rng.random(n) < sig(z)).astype(int)
    return np.clip(sig(3 * z), 1e-6, 1 - 1e-6), y          # overconfident scores

def ece(p, y, bins=10):
    idx = np.minimum((p * bins).astype(int), bins - 1)
    return sum(abs(p[idx == b].mean() - y[idx == b].mean()) * (idx == b).mean() for b in range(bins) if (
idx == b).any())

def fit_platt(s, y, iters=15):
    X = np.c_[logit(s), np.ones(len(s))]
    w = np.zeros(2)
    for _ in range(iters):
        p = sig(X @ w)
        H = X.T @ (X * (p * (1 - p))[:, None]) + 1e-6 * np.eye(2)
        w -= np.linalg.solve(H, X.T @ (p - y))
    return w

s_cal, y_cal = data(5000)
s_te, y_te = data(20000)
w = fit_platt(s_cal, y_cal)
p_te = sig(np.c_[logit(s_te), np.ones(len(s_te))] @ w)
print(f"ECE before={ece(s_te, y_te):.3f} after={ece(p_te, y_te):.3f}  slope={w[0]:.2f}")
bins = np.minimum((p_te * 5).astype(int), 4)
for b in range(5):
    print(f"bin {b}: mean p={p_te[bins == b].mean():.2f} observed={y_te[bins == b].mean():.2f}"
)
assert ece(p_te, y_te) < ece(s_te, y_te)
```

Split-conformal intervals for regression with a coverage check, and a classification abstention curve.

```
import numpy as np

rng = np.random.default_rng(0)
n = 6000
x = rng.uniform(-2, 2, n)
y = 2 * x + rng.normal(0, 0.5 + 0.3 * np.abs(x))
tr, cal, te = np.arange(n) < 2000, (np.arange(n) >= 2000) & (np.arange(n) < 4000), np.arange
(n) >= 4000

coef = np.polyfit(x[tr], y[tr], 1)
pred = np.polyval(coef, x)
alpha = 0.1
resid = np.sort(np.abs(y[cal] - pred[cal]))
k = int(np.ceil((len(resid) + 1) * (1 - alpha)))
q = resid[min(k, len(resid)) - 1]
cover = np.mean(np.abs(y[te] - pred[te]) <= q)
print(f"interval half-width={q:.2f} empirical coverage={cover:.3f} (target {1 - alpha})")
assert abs(cover - (1 - alpha)) < 0.03

p = rng.beta(0.6, 0.6, 20000)                    # calibrated probabilities of the positive class
label = (rng.random(len(p)) < p).astype(int)
conf, correct = np.maximum(p, 1 - p), ((p >= 0.5) == label)
order = np.argsort(-conf)
for cov in (1.0, 0.8, 0.6, 0.4):
    keep = order[: int(cov * len(p))]
    print(f"coverage={cov:.1f} error among answered={1 - correct[keep].mean():.3f}")
```

Interview Angle

Q: Your model is accurate but probabilities are off. Why does it matter and how do you fix it? **A:** Accuracy ignores probability quality, but downstream decisions use probabilities: expected-cost thresholds, risk pricing, ranking across models, human triage. Diagnose with a reliability table, ECE and Brier score on held-out data, then fit temperature, Platt or isotonic calibration on a separate calibration split, per segment if needed. Trade-off: isotonic is flexible but overfits small data. Check: ECE and per-segment reliability after calibration, re-measured after each retrain.

Q: Design an abstain-and-escalate policy for a medical triage assistant. **A:** Calibrate confidence, then set abstention thresholds by clinical cost: missed urgent cases are far costlier than extra reviews, so escalate on low confidence, out-of-distribution signals and any high-severity class regardless of confidence. Route to a clinician queue with capacity limits and monitor abstention rate by demographic group. Trade-off: safety versus reviewer load. Check: coverage-risk curve, recall on urgent cases, review queue latency, per-group abstention parity. Clinical use needs regulatory and clinical validation beyond this sketch.